STUDENT IMPLEMENTATION GUIDE

TrailBrief

Outdoor Trip Briefing Web Application

Build a web app that turns a location and trip date into a concise hiking briefing using official weather and terrain data, deterministic risk checks, and an evidence-grounded AI summary.

What the first version does

INPUT	GENERATED BRIEFING
Destination: choose a predefined location; coordinate entry in the API	Hourly weather for the selected local date
Trip date: within the available forecast window	Relevant NWS alerts and a clear data freshness label
Activity: day hiking	Elevation, risk flags, trip recommendation, packing checklist
Optional: hiking with a dog (Phase 2)	Each numeric fact links to its original source

A complete request, end to end

```
Browser form -> FastAPI -> NWS / USGS adapters -> PostgreSQL
             -> risk rules -> LLM text generation -> validation
             -> stored briefing -> React results page
```

Start here: five working days

01 Day 1 — Create the Git repository and project folders. Record three Southern California test locations with verified latitude/longitude. Write the one-page input/output contract.

02 Day 2 — Use the NWS API documentation to call /points/{latitude},{longitude}. Inspect the response and follow the forecastHourly URL it returns.

03 Day 3 — Write a Python function that extracts a trip day's hourly records (time, temperature, wind, precipitation probability). Preserve null values and units.

04 Day 4 — Start a FastAPI app with GET /health and GET /api/v1/weather?lat=...&lon=...&date=.... Return normalized JSON for a valid request.

05 Day 5 — Add pytest coverage for valid coordinates, invalid dates, upstream failure, missing precipitation probability, and local-time filtering.

Week 1 demo: run the backend locally, issue one request, show a real NWS source URL, and run the tests. The UI and LLM come later.

IMPLEMENTATION / PRODUCT

01 Product specification

One job: help a beginner prepare for a daytime hike. Start with Southern California; expand only after the first complete path works.

Input form

FIELD	V1 BEHAVIOR	VALIDATION
Destination	Choose among 3 verified locations	Store name, latitude, longitude, and timezone
Trip date	One day within returned NWS forecast horizon	Use destination-local date; disable unsupported dates
Activity	Day hike only	Fixed enum: day_hike
Dog / camping	Later milestone	Do not display unsupported pet restrictions

Results screen: required order

01 Status header — GO / RECONSIDER / AVOID / UNKNOWN, with short evidence-linked reasons. These labels are derived from code, not LLM judgment.

02 Weather — hourly temperature, wind, conditions, precipitation probability, and units. Missing values display “Not available.”

03 Alerts — NWS title, severity, effective/expiry times, and official URL. Clearly distinguish active alerts from forecasts for a future trip.

04 Terrain — elevation at the selected point from USGS, with units and point-level limitations.

05 Preparation — a concise summary and packing suggestions. Display origin of each fact; AI-generated guidance is visually distinct from retrieved data.

06 Sources / updated at — provider, fetch timestamp, relevant time window, and original link; allow the user to open source records.

Expected application states

STATE	UI EXAMPLE / EXPECTED BEHAVIOR
Loading	“Checking official forecasts and alerts…”; prevent duplicate submissions
Partial data	Weather visible but elevation unavailable; name missing source
Unsupported date	“NWS forecast is not available for this date”; no fabricated forecast
Provider timeout	Show incomplete status or retry option without claiming there are no alerts
LLM failure	Display rule-based facts and risk results; omit AI paragraph

Later features (not in the first working release)

Dog policies backed by individual trail/park sources; camping and 1–3 day dates; AQI, wildfire and road closures; destination search / map; saved trips and user feedback. Each additional data source requires a separate adapter and explicit coverage/quality logic.

IMPLEMENTATION / DESIGN

02 System architecture

Build one deployable web system, with separately testable components.

Recommended stack

LAYER	TECHNOLOGY	YOU WILL IMPLEMENT
Frontend	React + TypeScript + Vite	Form validation, async requests, report rendering, errors
Backend	Python + FastAPI + Pydantic	REST endpoints, adapter orchestration, schema checks
Persistence	PostgreSQL + SQLAlchemy + Alembic	Schema, migrations, query logic, backups
External data	HTTPX / Python	NWS hourly forecast, alerts, USGS elevation
AI	LLM API + structured response	Prompt, JSON validation, evidence checking, fallback
Runtime	Linux VM/VPS + Docker Compose + Nginx	Containers, networking, HTTPS, logs, deploy/restart

Repository layout

```
trailbrief/
  backend/
    app/api/          # HTTP routes and input validation
    app/providers/    # nws_weather.py, nws_alerts.py, usgs.py
    app/services/     # assemble.py, risks.py, llm.py
    app/db/           # models and database sessions
    migrations/       # Alembic migrations
    tests/            # unit, adapter, integration, evaluation
  frontend/src/       # pages, components, api client, types
  infrastructure/     # compose, nginx, deployment scripts
  docs/               # design notes and API examples
  fixtures/           # sanitized, versioned provider snapshots
```

Design boundaries

01 HTTP routes validate user input; they do not parse NWS responses directly.

02 Each provider adapter returns a predictable internal data structure with source metadata.

03 An assembly service combines provider results and persists the raw snapshots.

04 Risk rules read normalized data and return explicit rule IDs / evidence IDs.

05 The LLM receives the approved evidence set, creates explanatory text, and never replaces deterministic forecast or alert fields.

06 The frontend renders the validated response; it does not invent or infer risk thresholds.

Research exercise: compare running PostgreSQL in a Docker volume versus installing it directly on the VPS. Choose one approach and explain persistence, backup, and upgrade tradeoffs in docs/architecture.md.

IMPLEMENTATION / DATA

03 External-data adapters

Start with three API interactions; add providers only when the first three are testable.

Provider contracts

ADAPTER	OFFICIAL ENDPOINT / FIELDS	EDGE CASES
NWS hourly weather	/points/{lat},{lon} -> properties.forecastHourly; periods[] -> startTime, temperature, wind, precipitation probability	Weather grid mapping changes; missing fields; timezone; forecast horizon
NWS alerts	/alerts/active?point={lat},{lon}; event, severity, effective, expires, URL	No current alerts is not proof that a future trip has no risk
USGS elevation	EPQS point query with longitude x, latitude y, elevation units	Coordinates reversed, null response, measurement accuracy limits

Internal record: same pattern for every adapter

```
{
  "provider": "NWS",
  "status": "ok",
  "fetched_at": "2026-11-08T18:00:00Z",
  "source_url": "https://api.weather.gov/...",
  "valid_for": "2026-11-09",
  "data": { "hourly": [ ... ] },
  "error_code": null
}
```

The record above is a contract example, not an API response. Use timezone-aware datetimes and define what “valid_for” means for each source. A provider record can have status = ok, unavailable, or error; keep the difference.

Implementation steps

01 Create NWSWeatherProvider.fetch(lat, lon, date) and NWSAlertsProvider.fetch(lat, lon). Discover the forecast URL through /points; include an identifying User-Agent.

02 Convert source-specific structures into typed normalized fields. Keep raw source data and the original URL. Never convert unknown precipitation to 0%.

03 Create USGSElevationProvider.fetch(lat, lon); document longitude-versus-latitude order and unit conversions.

04 Add timeouts, bounded retries for transient failures, conditional caching based on provider semantics, and consistent error codes.

05 Save one sanitized successful response and one error fixture per provider. Write parser tests using fixtures so the test suite works without internet.

Freshness question to resolve in code

An alert returned as active now may not apply on a future trip date. Preserve effective and expiry times; display what the alert actually says and when it was retrieved. Do not silently convert “no active alerts now” into “no hazards on trip day.”

IMPLEMENTATION / BACKEND

04 Database and HTTP contracts

Define the smallest stable persistence model before wiring up the frontend.

PostgreSQL tables

TABLE	IMPORTANT COLUMNS	PURPOSE
briefing_requests	id UUID, lat, lon, local_date, activity, status, created_at	One user request and execution status
source_records	id, request_id, provider, status, fetched_at, source_url, raw_payload JSONB	Provider evidence, snapshots and failure records
briefings	id, request_id, verdict, reasons JSONB, weather JSONB, alerts JSONB, elevation_m, summary, created_at	Validated result returned to frontend

HTTP endpoints

METHOD + PATH	FUNCTION	RESPONSE
GET /health	Process health (no provider calls)	200 {"status":"ok"}
GET /api/v1/weather	Week 1 weather proof of concept	200 normalized hourly records
POST /api/v1/briefings	Create and compute a trip briefing	201 report or documented error
GET /api/v1/briefings/{id}	Retrieve a saved briefing	200 report / 404
GET /api/v1/briefings/{id}/sources	Retrieve evidence metadata / URLs	200 list of source records

POST request example

```
POST /api/v1/briefings
{
  "latitude": 34.2,
  "longitude": -118.1,
  "trip_date": "2026-11-12",
  "activity": "day_hike"
}
```

Response shape (illustrative)

```
{
  "id": "<uuid>", "status": "complete",
  "verdict": "reconsider",
  "reasons": [{"rule_id":"WIND_CAUTION", "evidence_ids":["nws-01"]}],
  "weather": {"hours": []}, "alerts": [],
  "elevation_m": null, "summary": "...",
  "sources": [{"id":"nws-01","url":"...","fetched_at":"..."}]
}
```

Write Alembic migrations and use foreign keys between all three tables. Test that an interrupted or failed request can be recorded and retrieved without leaving an invalid “complete” briefing.

IMPLEMENTATION / APPLIED AI

05 Decision engine and AI output

A complete briefing must be reproducible from the same saved evidence, even when the LLM is unavailable.

Decision logic: example workflow

01 Load normalized weather, alert, and elevation records for the request. Keep provider status and timestamps.

02 Run deterministic checks: unsupported date, missing critical weather data, applicable official warning, and project-defined wind / temperature thresholds.

03 Return verdict in {go, reconsider, avoid, unknown}, a list of triggered rule IDs, and the exact evidence IDs used.

04 Prepare an LLM input containing only validated facts, evidence IDs, activity type, and a request for summary and gear suggestions.

05 Validate output schema, evidence references, text constraints, and numeric claims. If invalid or timed out, show the rule-based briefing without AI text.

Structured AI output contract

```
{
  "overview": "string (short; no unsupported measurements)",
  "gear": [
    {"item":"string", "reason":"string", "evidence_ids":["nws-01"]}
  ],
  "limitations": ["string"]
}
```

Do not ask the LLM to regenerate weather numbers, change warning status, or choose the final verdict. Weather tables and source links should be rendered directly from normalized backend data.

Evaluation dataset

GROUP	COUNT	EXPECTED CHECK
Normal trips	20	No fabricated hazards; grounded evidence
Caution conditions	15	Reconsider + matching trigger/evidence
High-risk conditions	10	Avoid + applicable blocking rule
Missing/error cases	5	Unknown or documented partial response; no false certainty

Freeze provider snapshots and expected rule results in fixtures. Measure verdict exact-match, risk recall, evidence-reference validity, schema pass rate, failure-handling coverage, and end-to-end latency. Record actual results rather than predetermined improvements.

IMPLEMENTATION / ROADMAP

06 Twelve-week build schedule

The milestones below are ordered so that each new layer is built on working software from the previous stage.

MILESTONE	WEEKS	FINISHED RESULT
01 Weather proof of concept	1	NWS data through a tested FastAPI endpoint
02 Backend and database	2	POST/GET briefing contract with persisted records
03 Source integration	3–4	NWS hourly + alerts + USGS adapters, error states and caching
04 Risk assessment	5	Deterministic verdict, sources, persistent report assembly
05 AI and evaluations	6–7	Grounded JSON summary, validator, repeatable 50-case suite
06 User-facing application	8	Functional React form, results page, and source links
07 Tests and deployment	9–10	CI checks and deployed VPS-backed application
08 Release and iteration	11–12	Measured reliability, operational checks, user feedback and release

How to progress through a milestone

Implement the tasks in order; check the “Done when” items using a local run, an API request, or an automated test. Keep a short CHANGELOG entry for decisions that alter an API or data contract. If a third-party source fails, use stored fixtures to continue the feature work rather than inventing data.

Practical weekly rhythm

01 Early week: inspect documentation and write down the expected input/output.

02 Midweek: implement the vertical slice, including one normal path and one failure path.

03 End of week: run tests, demo the feature locally or on the VPS, and note the next known issue.

The first complete browser → API → database → provider → result flow is more useful than connecting ten providers that cannot be demonstrated together.

MILESTONE 01 / WEEK 1

Weather API proof of concept

Make a real weather request and expose typed hourly data through a local API before writing any frontend.

Build steps

01 Initialize Git repository, Python environment and FastAPI project. Commit a working GET /health endpoint.

02 Choose three test destinations and verify coordinates; prepare a simple config file holding the destination names.

03 Read the NWS /points endpoint response; follow properties.forecastHourly and extract weather periods for one local date.

04 Create Pydantic models for HourlyWeather and WeatherResponse; make GET /api/v1/weather return the normalized structure.

05 Write pytest tests with saved source responses. Cover invalid lat/lon, unsupported date, null precipitation, and upstream timeout.

Implementation details to work out

- Understand why /points must be called before forecastHourly and why a grid mapping can change.
- Parse timezone-aware startTime values; do not filter hours using UTC date by mistake.
- Check status codes and missing keys; do not assume all weather fields are always present.

Done when

- GET /health returns 200 and a valid /api/v1/weather request returns real hourly data.
- Three automated cases pass without network access by using saved fixtures.
- One demo shows the public forecast URL and a malformed-input error response.

If time remains

Add an HTTP caching header only after measuring whether it is necessary.

MILESTONE 02 / WEEK 2

FastAPI and PostgreSQL foundation

Create the API contract and persistent request lifecycle. Weather may be stubbed with fixtures while the database path is implemented.

Build steps

01 Draw a minimal ER diagram for briefing_requests, source_records, and briefings; choose column types and constraints.

02 Install PostgreSQL locally. Implement SQLAlchemy models, session lifecycle, and the first Alembic migration.

03 Implement POST /api/v1/briefings. Validate coordinates, local date, and activity; create a request row and a fixture-backed briefing.

04 Implement GET /api/v1/briefings/{id} and /{id}/sources; use IDs, foreign keys, and consistent 404/422 errors.

05 Write integration tests using a temporary test database; verify data persists across process restarts.

Implementation details to work out

- Decide whether IDs use UUIDs or generated integers and why.
- Separate API schemas from database models to avoid leaking internal columns.
- Learn when to commit or roll back a transaction and how to represent failed requests.

Done when

- A POST creates a database-backed briefing and GET retrieves the same stored record.
- Migration creates all required tables on a fresh database.
- Invalid input cannot create a completed report.

If time remains

Add pagination if saved briefing retrieval starts to list many records.

MILESTONE 03 / WEEK 3–4

Reliable official-data pipeline

Replace fixture-only data with three real providers and keep the app useful when one provider is missing.

Build steps

01 Implement NWS weather adapter and NWS active-alert adapter as separate modules with a common result shape.

02 Implement USGS elevation point query. Normalize elevation to meters; preserve returned unit and source details.

03 Save raw provider snapshots, normalized results, fetch timestamps, status, and source URLs in PostgreSQL.

04 Add bounded timeouts and retries, including a cache policy for repeated location/date queries.

05 Create adapter tests for null values, invalid coordinates, provider 429/500 responses, empty alerts, and stale cache.

Implementation details to work out

- Treat active alerts as a time-stamped observation; inspect effective/expires times when relating them to a future trip.
- Write per-provider normalization functions before joining records together.
- Do not let a USGS failure erase a valid weather forecast; return a partial result.

Done when

- One API call returns weather, alerts, elevation, and source freshness fields.
- A forced elevation outage still produces a partial briefing rather than a fabricated value.
- Repeated requests visibly use the cache; provider fixtures can reproduce both success and error responses.

If time remains

Only after these adapters pass tests, consider adding AirNow AQI as a fourth provider.

MILESTONE 04 / WEEK 5

Risk rules and report assembly

Turn normalized provider facts into a transparent report with deterministic recommendations.

Build steps

01 Define and implement RiskRule as an interface/function taking typed evidence and returning rule_id, severity, and evidence IDs.

02 Start with official NWS alerts, unavailable-critical-data handling, and clearly documented project thresholds for weather conditions.

03 Implement the final verdict precedence: blocking rule > caution rule > normal; missing critical evidence gives unknown.

04 Create an assembly service that builds one report object and stores rule outputs, facts, and source records.

05 Write unit tests that change exactly one evidence value and assert the verdict and rule ID.

Implementation details to work out

- Separate “app heuristic threshold” from official NWS warning severity in code and UI text.
- Choose deterministic rule priority and document how overlapping warnings are handled.
- Store the evidence snapshot used for each decision so a result can be reproduced later.

Done when

- Briefing generation works without any LLM call.
- A blocking-warning fixture cannot produce a GO verdict.
- Missing required weather returns UNKNOWN, not “no risk.”

If time remains

Add a rules version field so older generated briefings retain the version used.

MILESTONE 05 / WEEK 6–7

Grounded AI summaries and evaluation

Add human-readable explanations without letting the language model replace source facts or rule decisions.

Build steps

01 Write a narrow LLM response model: overview, gear suggestions, limitations, and evidence IDs; no duplicate weather table fields.

02 Build the prompt from normalized evidence and rule outputs. Include report source IDs; request strict structured output.

03 Implement output validation: schema check, valid evidence IDs, length limits, and numeric-claim handling.

04 Handle API timeout, invalid JSON and unsupported citations by dropping the AI section while retaining the factual briefing.

05 Create 50 frozen evaluation cases with expected verdicts and warnings. Write a repeatable evaluation runner and result summary.

Implementation details to work out

- Compare a baseline prompt with one revised prompt using exactly the same stored fixtures.
- Track provider failures separately from model output failures to avoid misleading scores.
- Store a prompt/model version and execution latency, but never store credentials in report snapshots.

Done when

- LLM-generated text is stored separately from verified weather and warning fields.
- Every evidence ID refers to a saved source record; malformed outputs fail validation.
- The evaluation command prints measured verdict accuracy, recall, schema pass rate and evidence-reference validity.

If time remains

Try two model configurations and explain the quality/cost/latency tradeoff using collected results.

MILESTONE 06 / WEEK 8

React interface and complete user journey

Connect the browser to the real backend so a user can create, inspect, and revisit a briefing.

Build steps

01 Create a destination/date form in React + TypeScript with required-field and date validation.

02 Build a typed API client and request states: idle, loading, complete, partial, failed.

03 Render verdict, hourly forecast, alerts, elevation, AI summary, packing list, and original source links.

04 Show provider timestamps, source availability and unknown values; keep each section independently usable.

05 Implement accessible form labels, keyboard focus, loading indicators and mobile-width layout.

Implementation details to work out

- Use stable TypeScript types generated or manually matched to the FastAPI response schema.
- Use a local development proxy; understand CORS rather than disabling all browser protections.
- Test a response where elevation exists but AI text does not, and one where weather is unavailable.

Done when

- A new user can create a briefing from the browser and revisit it by a stable URL.
- A partial data response renders without a blank page or misleading zero values.
- The mobile-sized page remains readable and every source opens the original provider.

If time remains

Add a small feedback control only after end-to-end flow is stable.

MILESTONE 07 / WEEK 9–10

Automated tests and Linux deployment

Run the complete app on a student-administered Linux VM with persistent data, HTTPS and repeatable deployment.

Build steps

01 Add backend unit/integration tests, frontend component tests and at least one browser end-to-end test.

02 Provision a small Linux VPS, create a non-root deploy user, set up SSH keys and firewall rules.

03 Build the frontend and backend for production; deploy with Docker Compose or another documented self-managed runtime.

04 Configure Nginx to serve the frontend and reverse-proxy /api; configure a domain and HTTPS certificate.

05 Run database migrations during deployment; verify persistent Postgres storage and restore a test backup.

06 Create CI checks for tests and a deployment runbook describing restart, logs, rollback and troubleshooting.

Implementation details to work out

- Know which ports are public; the database should not be publicly reachable.
- Keep API keys in deployment environment configuration, not browser code or Git.
- Compare local and production configuration and document why they differ.

Done when

- Public HTTPS URL serves the full trip briefing flow.
- Recreating app containers does not delete database records.
- A clean backup restores into a fresh test database and CI runs the test suite.

If time remains

Add uptime monitoring and alerting after confirming health checks and logs work.

MILESTONE 08 / WEEK 11–12

Evaluation, reliability and release

Measure how the deployed system behaves; fix the highest-impact failures and publish a usable release.

Build steps

01 Run the frozen evaluation suite against final rules and model pipeline. Record real metrics with sample sizes.

02 Run repeated API requests to estimate p50/p95 latency and compare cache hit versus miss.

03 Test real failure modes: NWS timeout, expired data, broken LLM response, database restart, invalid input.

04 Review logs, fix high-severity defects, and verify a redeploy and rollback sequence.

05 Run a few structured user walkthroughs; record actual tasks completed and problems observed.

06 Finalize README: setup, demo URL, architecture, API examples, evaluation method, known limitations and screenshots.

Implementation details to work out

- Keep “no alerts returned,” “provider unreachable,” and “no risk identified” distinct in reports and documentation.
- Measure from production logs or a repeatable local tool, not by guessing performance.
- Describe one design tradeoff you changed based on testing or feedback.

Done when

- An external user can open the deployed app and complete the briefing flow.
- Test/evaluation results and limitations are documented from real runs.
- A short demo can trace one number from source response through storage to UI.

If time remains

Choose one Phase 2 feature and write a scoped technical design instead of beginning a large rewrite.

IMPLEMENTATION / RESOURCES

07 Reference links and starting checklist

Read these official references while building

RESOURCE	WHY IT MATTERS
NWS API — weather.gov/documentation/services-web-API	/points discovery, hourly forecast, User-Agent, caching
NWS alerts — weather.gov/documentation/services-web-alerts	Warnings, active-alert semantics and usage
USGS EPQS — epqs.nationalmap.gov/v1/docs	Elevation query fields and units
FastAPI — fastapi.tiangolo.com	Routes, request/response models, automatic OpenAPI docs
PostgreSQL — postgresql.org/docs/	Tables, constraints, JSONB, transactions, backup/restore
Alembic — alembic.sqlalchemy.org	Database migration workflow
React — react.dev / TypeScript — typescriptlang.org/docs/	UI component structure and typed data
Docker Compose — docs.docker.com/compose/	Multi-container startup and persistent volumes
Nginx — nginx.org/en/docs/	Reverse proxy and serving static frontend files
Certbot — certbot.eff.org	HTTPS certificate issuance/renewal

The first commit should contain

- backend/ FastAPI skeleton with /health; tests/ directory and dependency file
- docs/product.md with the exact first-version inputs and result sections
- docs/data-sources.md with NWS endpoints and the sample point used
- README.md with local setup, one curl example and the current known limitation
- .gitignore and .env.example (placeholder names only)

First API experiment to investigate

```
GET https://api.weather.gov/points/34.2,-118.1
Header: User-Agent: TrailBrief (your-contact@example.com)
Then request the forecastHourly URL returned by the point lookup.
Inspect properties.periods[].startTime and the trip-date timezone.
```

Verify the coordinates and the returned forecast coverage before using them in the application. Follow the provider documentation for content types, request pacing and error behavior.